Are You Getting What You Pay For?

Auditing Model Substitution in LLM APIs: Why Software Tests Fail and Hardware Attestation Wins

Commercial LLM APIs may silently serve cheaper models — current detection methods are fundamentally broken.

arxiv.org/abs/2504.04715

KEY FINDING

Providers face strong economic incentives to covertly replace advertised models — users have no verification mechanism in black-box APIs.

The Model Substitution Problem: Economic Incentives Drive Covert Swaps

SUBSTITUTION TYPES

Smaller Model
Serve 8B parameters instead of advertised 70B

Dramatically lower GPU cost per inference call

Quantized Variant
FP8 / INT8 precision instead of FP16

Reduces memory footprint; near-identical benchmark scores

Fine-tuned / Updated Version
Modified weights silently deployed mid-contract

Cost or capability optimization without user notification

Entirely Different Model Family
Different architecture, training data, and distribution

Hardest to detect; may differ on tail behaviors only

FORMAL FRAMEWORK

Hypothesis Testing Problem

H_0: API serves P_spec(y|x) — advertised model

H_1: API serves P_actual(y|x) — substitute model

Auditor observes only text outputs y given prompts x

Economic Driver
GPU cost savings from cheaper model inference while charging premium prices

No Verifiability
Black-box API: users observe only outputs no weight or code access

THE CORE CHALLENGE

API outputs are the only observable signal — and for quantized or randomized substitutions, that signal is nearly indistinguishable.

Detection Challenge by Substitution Type

- Smaller model (8B vs 70B): detectable via benchmark — but slow and costly
- Quantized variant (FP8): nearly undetectable — scores within error bars
- Fine-tuned version: depends on magnitude of update
- Different family: detectable in theory, but query-intensive in practice

KEY FINDING

FP8 quantization barely degrades benchmark scores — differences often fall within error bars, making detection extremely challenging.

FP8 Quantization Preserves Most Accuracy — Making Substitution Hard to Detect

BENCHMARK ACCURACY: ORIGINAL vs FP8 QUANTIZED (with ± std dev)

MODEL	MMLU	GSM8K	MATH	GPQA Diamond
LLAMA-3-8B INSTRUCT				
Llama-3-8B-Instruct (FP16)	**62.69** ± 0.18	**61.14** ± 3.47	**20.65** ± 3.43	**22.62** ± 0.22
Llama-3-8B-Instruct-FP8	62.43 ± 0.26	60.90 ± 4.10	14.91 ± 2.49	20.14 ± 0.24
LLAMA-3-70B INSTRUCT				
Llama-3-70B-Instruct (FP16)	**78.05** ± 0.08	**88.06** ± 1.44	**35.69** ± 1.33	**29.60** ± 0.51
Llama-3-70B-Instruct-FP8	77.88 ± 0.13	87.35 ± 1.24	35.75 ± 1.16	33.12 ± 0.30
GEMMA-2-9B				
Gemma-2-9b-it (FP16)	**71.86** ± 0.08	**81.80** ± 1.35	**33.41** ± 0.28	**28.64** ± 2.97
Gemma-2-9b-it-FP8	71.92 ± 0.11	79.41 ± 1.14	32.53 ± 0.34	27.81 ± 3.22
QWEN2-72B				
Qwen2-72B-Instruct (FP16)	**82.18** ± 0.08	**86.72** ± 1.00	**37.39** ± 1.41	**29.93** ± 2.71
Qwen2-72B-Instruct-FP8	81.98 ± 0.08	86.82 ± 0.97	37.67 ± 1.39	31.08 ± 1.96
MISTRAL-7B				
Mistral-7B-Instruct-v0.3 (FP16)	**59.15** ± 0.10	**35.90** ± 4.54	**8.94** ± 1.24	**21.60** ± 0.17
FP8: 58.77±0.13	58.77	32.20	7.68	22.72

= FP8 quantized variant

Red = notable drop beyond error bars

Most FP8 scores fall within original ± std dev — making detection nearly impossible

Source: arxiv.org/abs/2504.04715

KEY FINDING

Higher temperatures degrade all models uniformly — amplifying noise rather than revealing substitution signals.

Higher Temperature Amplifies Performance Gaps but Increases Noise

BENCHMARK ACCURACY vs TEMPERATURE — FIVE MODELS, FOUR TASKS

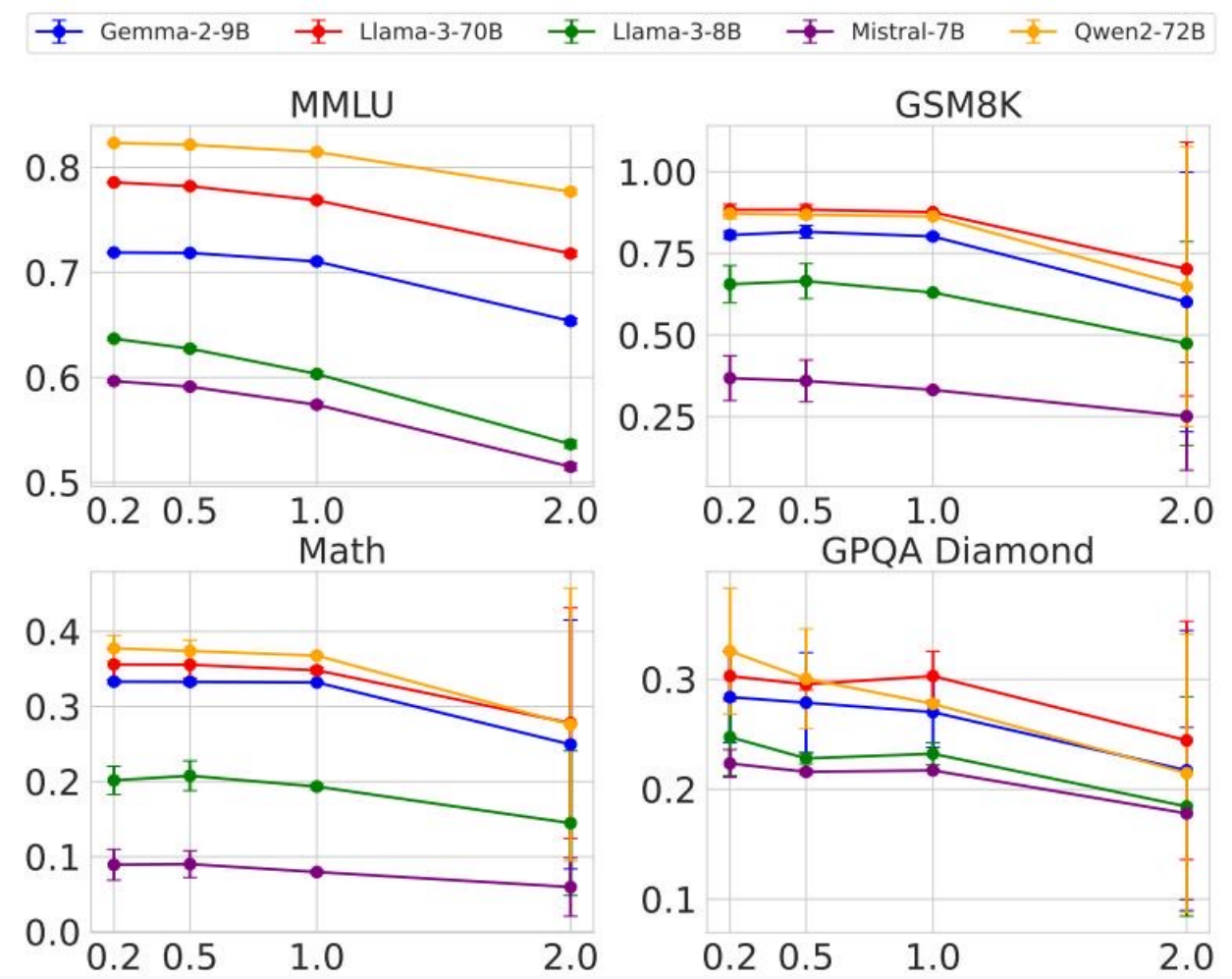


Figure 1: Benchmark accuracy vs temperature (T=0.2, 0.5, 1.0, 2.0) across five model families and four tasks

ANALYSIS

Universal Degradation at T=2.0

At temperature 2.0, all five model families show significant performance collapse across all four benchmarks.

Original and FP8 variant both degrade equally

Detection Signal Drowned by Noise

The performance gap between original and FP8 variant narrows or becomes noise-dominated at high temperatures, not wider.

Higher T does not help auditors distinguish substitution

Temperatures Tested

T = 0.2 · 0.5 · 1.0 · 2.0

Five model families · Four benchmarks

(MMLU, GSM8K, MATH, GPQA Diamond)

Implication for Auditors

Temperature manipulation cannot improve statistical detection power. The substitution signal remains buried in variance.

Source: arxiv.org/abs/2504.04715, Figure 1

KEY FINDING

Production inference nondeterminism renders log-probability fingerprinting unreliable — legitimate variance is indistinguishable from substitution.

Log Probability Fingerprinting Fails Due to Inference Nondeterminism

LLAMA-3-8B: LOG-PROB VARIATION ACROSS FRAMEWORKS AND HARDWARE

Model	BERT Acc	T5 Acc	GPT2 Acc	LLM2Vec Acc
Llama3-70B-Instruct-FP8	50.55	50.10	50.45	49.90
Llama3-70B-Instruct-INT8	51.60	49.90	51.30	50.25
Gemma2-9b-it-FP8	49.95	50.30	51.20	49.80
Gemma2-9b-it-INT8	49.00	49.70	51.65	49.55
Mistral-7b-v3-Instruct-FP8	50.55	49.75	48.70	48.75
Mistral-7b-v3-Instruct-INT8	49.70	50.75	50.50	51.15
Qwen2-72B-Instruct-FP8	50.05	50.25	48.75	49.80
Qwen2-72B-Instruct-INT8	50.75	50.55	49.45	50.20

Figure 5: Log-probability variation for the same Llama-3-8B model across different inference frameworks and GPU hardware

Software Nondeterminism

The same Llama-3-8B model weights produce measurably different log probabilities depending on the inference framework version:

- transformers 4.40 vs transformers 4.50
- vLLM 0.6.2 vs vLLM 0.8.2

Framework version updates introduce floating-point arithmetic divergence even with identical weights

Hardware Nondeterminism

Different GPU microarchitectures produce different log-probability distributions for identical model weights and inference code:

- NVIDIA A100 vs NVIDIA H100
- Tensor core implementations differ

An auditor's local A100 reference cannot reliably compare against a provider's H100 deployment

Source: arxiv.org/abs/2504.04715, Figure 5

KEY FINDING

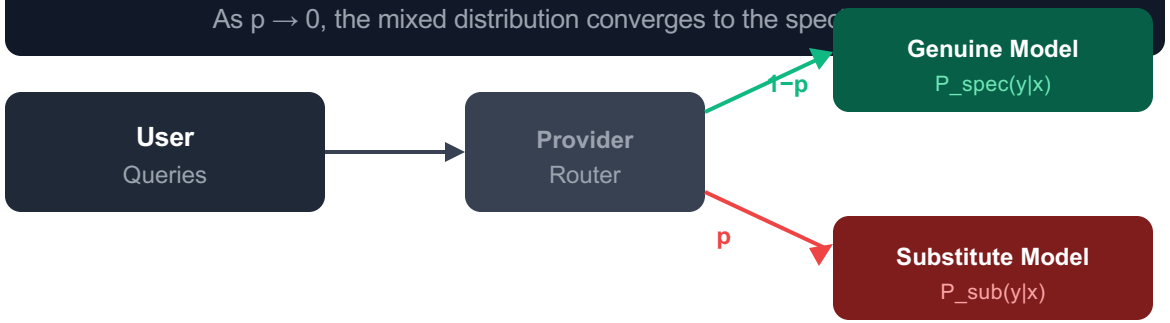
Mixing substituted responses at low probability p → 0 makes statistical detection query-prohibitive; adversarial providers can always evade below the detection threshold.

Randomized Routing at Low Rates Makes Statistical Detection Query-Prohibitive

RANDOMIZED SUBSTITUTION MODEL

$$P_{mix}(y|x) = (1-p)\cdot P_{spec} + p\cdot P_{sub}$$

As p → 0, the mixed distribution converges to the spec



Query Budget Required for Reliable Detection

Substitution Rate (p)	Approx. Queries Needed	Feasibility
p = 0.5 (50% substitute)	~100 queries	Possible
p = 0.1 (10% substitute)	~10,000 queries	Expensive
p = 0.01 (1% substitute)	~1,000,000 queries	Infeasible
p → 0 (near-undetectable)	→ ∞	Impossible

Query budget grows as $O(1/p^2)$ — providers can always choose p small enough to defeat any fixed query budget.

ADVERSARIAL COUNTERMEASURES

Randomized Substitution

Route only a fraction p of user queries to the cheaper substitute model, while serving the genuine model for the remaining (1−p) fraction.

As p decreases, the mixed distribution P_mix converges to P_spec — making the substitution statistically indistinguishable from serving the genuine model.

Result: Requires exponentially more queries for detection.

Benchmark Evasion

Detect known benchmark prompts (MMLU, GSM8K, MATH, etc.) and route them to the genuine model, while serving substitutes for regular traffic.

Benchmark prompts have known signatures and can be fingerprinted by semantic similarity or exact match — making audit-specific queries ineffective.

Result: Audit traffic gets genuine model; real traffic gets substitute.

CONCLUSION

All three software-only auditing approaches share a fatal flaw: they are defeated by the very properties of production LLM deployment.

Software-Only Auditing Methods Are Fundamentally Unreliable

FAILURE MODE ANALYSIS MATRIX

METHOD TYPE	CORE MECHANISM	FATAL WEAKNESS	PRACTICAL LIMITATION
Text-Based Statistical Tests e.g., Maximum Mean Discrepancy (MMD) Text output distribution	Compare text output distributions between a local reference model and the API responses using hypothesis tests (e.g., MMD). Requires large sample sizes to reach statistical significance.	**Cannot detect subtle variants** FP8 quantization keeps output distribution nearly identical — differences fall within noise. Randomized substitution at small p is undetectable.	Requires 10,000–1,000,000+ API queries — cost-prohibitive for continuous monitoring. **Not practical at provider scale.** Even large budgets defeated by randomized routing.
Log-Probability Fingerprinting Token-level log-prob comparison via API Metadata-based method	Compare token-level log-probs from a local reference model against API log-prob outputs. Uses fine-grained probability distributions as a "fingerprint" of the model's identity.	**Production nondeterminism** Same weights produce different log-probs on: transformers 4.40 vs 4.50, vLLM 0.6.2 vs 0.8.2, A100 vs H100 GPUs. Legitimate variance ≈ substitution signal	Auditor's local reference cannot match provider infrastructure — different hardware guarantees different log-prob values. **Cannot distinguish infrastructure variation from substitution.**
Adversarial Countermeasures Active provider-side evasion attacks Not a detection method	Provider actively detects and evades audit attempts: benchmark evasion + randomized routing. Targets the detection method itself rather than the model distribution.	**Undermines all methods** Benchmark prompts can be fingerprinted; audit-specific traffic routes to genuine model. No software method is immune to adversarial adaptation.	Any query-based detection can be actively circumvented by a motivated provider. **Adversarial game: defender always at a structural disadvantage** in black-box API settings.

CONCLUSION: No software method provides reliable, query-efficient model integrity verification under adversarial conditions.
Each approach fails independently, and adversarial countermeasures defeat them collectively.

SOLUTION

Trusted Execution Environments provide hardware-level cryptographic proof of model identity — the only currently deployable solution with provable guarantees.

Trusted Execution Environments Provide Provable Cryptographic Model Integrity

Software Auditing

Fundamentally Unreliable

Statistical text tests
Require 10,000–1M+ queries.
Fail against FP8 and randomized substitution.

Log-prob fingerprinting
Defeated by software version and GPU hardware nondeterminism in production.

Adversarial evasion always possible
Providers can fingerprint audit queries and apply benchmark evasion.

Zero-Knowledge Proofs (ZKPs)
Computationally prohibitive for LLM scale.
Not deployable today for 70B+ models.

VERDICT: No software method can provide cryptographic, tamper-proof guarantees.

TEE Hardware Attestation

Provably Secure

Cryptographic Attestation
Hardware-level proof that specific model weights are loaded and inference code is running correctly.

NVIDIA Confidential Computing (H100)
TEE support on H100 GPUs is deployable today.
Provider runs inference inside a secure enclave.

Only Modest Performance Overhead
Unlike ZKPs, TEEs introduce only small latency increases — practical for production deployment.

Defeats All Substitution Types
Smaller models, FP8 variants, fine-tuned versions, and different families — all verifiable via attestation.

VERDICT: TEEs are provable, efficient, and robust — the only currently viable path to verifiable LLM APIs.

Source: arxiv.org/abs/2504.04715

Key Takeaways: Close the Verification Gap with Hardware Attestation

1 **Model substitution is economically motivated and nearly undetectable via software**

FP8 quantization cuts GPU costs significantly while keeping benchmark scores within error bars of the original model.

Providers have clear financial incentives; users have zero visibility into what is actually being served.

2 **Statistical text tests (e.g., MMD) require impractically large query budgets**

Detection requires 10,000–1,000,000+ queries; randomized routing at small p makes any fixed budget insufficient.

Query budget grows as $O(1/p^2)$ — providers can always choose p small enough to remain undetected.

3 **Log-probability fingerprinting is defeated by production-level nondeterminism**

Same weights produce different log-probs on transformers 4.40 vs 4.50, vLLM 0.6.2 vs 0.8.2, A100 vs H100.

Auditors cannot distinguish legitimate infrastructure variation from actual model substitution.

4 **Randomized routing and benchmark evasion actively undermine all software detection**

Motivated providers can fingerprint audit traffic and apply benchmark evasion, making audits trivially defeatable.

This is an adversarial game where defenders are at a structural disadvantage in black-box API settings.

5 **Trusted Execution Environments are the only currently viable path to verifiable LLM APIs**

TEEs provide cryptographic attestation of model weights and inference code — NVIDIA H100 support is deployable today.

Provable, efficient (modest overhead), and robust against all substitution types.

CALL TO ACTION

The ML community should demand TEE-backed attestation as a standard requirement in commercial LLM API contracts.
Model integrity guarantees should be cryptographic, not contractual.

arxiv.org/abs/2504.04715 · Auditing Model Substitution in LLM APIs